

PYTHON 教學系列 • CLAUDE-AGENT-SDK 0.2.89

Claude Agent SDK

把 Claude Code 當成程式庫，打造會自己讀檔、執行指令、搜尋、
改程式碼的 AI Agent。

19

堂課

5

階段

Py

3.10+



WHAT IS IT

這個 SDK 是什麼

▶ 內建 agent 迴圈

讀檔、跑指令、搜尋、改程式碼，開箱即用

▶ 對比 Client SDK

那邊你自己寫工具迴圈；這邊 Claude 幫你做掉

▶ 同一套 Claude Code 核心

工具、上下文管理、權限全都在

main.py

```
async for message in query(  
    prompt="找出 auth.py 的 bug 並修好",  
    options=ClaudeAgentOptions(  
        allowed_tools=["Read", "Edit", "Bash"],  
    ),  
):  
    print(message)    # 讀檔、找 bug、改檔，全自動
```

TWO ENTRY POINTS

兩種進入點

▶ **query()**

函式、單次、用完即走——一次性任務、批次腳本

▶ **ClaudeSDKClient**

類別、持久連線、多輪對話——互動式 App

▶ **共用 ClaudeAgentOptions**

同一套設定，兩種節奏

main.py

```
# 單次：用完即走
async for m in query(prompt="..."):
    handle(m)

# 多輪：跨輪記得前文
async with ClaudeSDKClient() as client:
    await client.query("第一輪")
    async for m in client.receive_response():
        handle(m)
    await client.query("第二輪")
```

19 課 • 5 階段

① 基礎

- 01 Hello World
- 02 訊息型別
- 03 即時串流

④ 子代理

- 10 自訂子代理
- 11 工具範圍
- 12 專屬 MCP
- 13 子代理追蹤

② 工具與權限

- 04 權限控制
- 05 自訂工具
- 06 Hooks
- 07 外部 MCP

⑤ 生產化

- 14 成本可觀測性
- 15 OTel 遙測
- 16 執行限制
- 17 安全沙箱
- 18 檔案系統記憶
- 19 進階持久化

③ 設定與狀態

- 08 系統提示 • 模型
- 09 Session 持久化

SETUP

跑起來需要什麼

- ▶ **Python 3.10+ 與 Node.js**

SDK 底層會啟動 Claude Code CLI 子程序

- ▶ **一行安裝**

`pip install claude-agent-sdk`

- ▶ **設定認證**

`export ANTHROPIC_API_KEY`，或走 Bedrock / Vertex / Azure

● ● ● shell

```
pip install claude-agent-sdk
```

```
export ANTHROPIC_API_KEY=sk-ant-...
```

```
python 01_hello_world/main.py
```

PART 1 • 基礎

基礎

先搞懂怎麼送請求、怎麼讀回來的訊息流、怎麼即時串流。

01 Hello World

02 訊息型別

03 即時串流

QUERY()

Hello World

整個 SDK 最基礎的形狀：送 prompt → 迭代訊息流 → 挑出要的。

- ▶ **query() 回傳訊息流**

不是一句字串，而是一串可 async 迭代的訊息

- ▶ **認出 AssistantMessage**

走訪 content，挑 TextBlock 拿文字

- ▶ **ResultMessage 收尾**

最後一則，帶成本與回合數

別期待 query() 直接給字串——文字藏在 AssistantMessage 的 TextBlock 裡。

Hello World

送一句話，取出文字與成本

```
main.py

async for message in query(
    prompt="What is 2 + 2?",
    options=ClaudeAgentOptions(max_turns=1),
):
    if isinstance(message, AssistantMessage):
        for block in message.content:
            if isinstance(block, TextBlock):
                print(block.text)
    elif isinstance(message, ResultMessage):
        print(f"${message.total_cost_usd:.4f}")
```

訊息型別

一個會用工具的 agent，吐出的是一齣有來有回的戲。

- ▶ **SystemMessage(init)**

開場：session_id、可用工具、模型

- ▶ **AssistantMessage**

Claude 發言：TextBlock 或 ToolUseBlock

- ▶ **UserMessage / ResultMessage**

工具結果回填；最後總結成本與回合

isinstance 分流，是讀訊息流的標準姿勢。

訊息型別

用 isinstance 把每種訊息分流

● ● ● main.py

```
if isinstance(message, SystemMessage):
    print("init", message.data["session_id"])
elif isinstance(message, AssistantMessage):
    for block in message.content:
        if isinstance(block, ToolUseBlock):
            print("呼叫", block.name, block.input)
elif isinstance(message, ResultMessage):
    print(message.subtype, message.total_cost_usd)
```

INCLUDE_PARTIAL_MESSAGES

即時串流

打開一個開關，從「整段等完」變「逐字到達」。

- ▶ **include_partial_messages**

打開後額外收到一串 StreamEvent

- ▶ **解析 text_delta**

從 event 取出每次新增的文字片段

- ▶ **flush 即時輸出**

串起來就是打字機效果

StreamEvent 是額外的——完整的 AssistantMessage 仍會照常收到。

即時串流

逐 token 印出，做出打字機效果

main.py

```
options = ClaudeAgentOptions(include_partial_messages=True)

async for message in query(prompt="...", options=options):
    if isinstance(message, StreamEvent):
        e = message.event
        if e.get("type") == "content_block_delta":
            text = e["delta"].get("text", "")
            print(text, end="", flush=True)
```

工具與權限

內建工具的權限把關、寫自己的工具、攔截行為、接外部 MCP。

04 權限控制

05 自訂工具

06 Hooks

07 外部 MCP

CAN_USE_TOOL

權限控制

allowed_tools 只看工具名；can_use_tool 看實際參數臨場決定。

- ▶ **放行 / 拒絕 / 改寫**

Allow、Deny、Allow(updated_input=...)

- ▶ **updated_input 是殺手鐮**

把危險參數消毒後再放行

- ▶ **permission_mode="default"**

用 default，callback 才會被呼叫

權限控制

依工具與輸入動態守門

main.py

```
async def gatekeeper(tool, args, ctx):
    if tool in {"Read", "Glob", "Grep"}:
        return PermissionResultAllow()
    if tool == "Bash" and "rm -rf" in args.get("command", ""):
        return PermissionResultDeny(message="禁止危險指令")
    return PermissionResultAllow()

options = ClaudeAgentOptions(
    can_use_tool=gatekeeper, permission_mode="default")
```

自訂工具

把 Python 函式變成 Claude 的工具：兩個裝飾器 + 一次打包。

- ▶ **@tool 標記函式**
名稱、給模型看的說明、參數 schema
- ▶ **create_sdk_mcp_server 打包**
進程內 MCP，不必另開程序
- ▶ **工具全名 mcp__server__tool**
用全名在 allowed_tools 放行

回傳格式固定：{'content': [{'type': 'text', 'text': ...}]}

自訂工具

定義工具、打包成進程內 MCP 伺服器

main.py

```
@tool("add", "把兩個數字相加", {"a": float, "b": float})
async def add(args):
    total = args["a"] + args["b"]
    return {"content": [{"type": "text", "text": str(total)}]}

server = create_sdk_mcp_server(name="toolbox", tools=[add])
options = ClaudeAgentOptions(
    mcp_servers={"toolbox": server},
    allowed_tools=["mcp__toolbox__add"],
)
```

生命週期 Hooks

在 agent 生命週期的多個時間點，插入你的程式碼。

- ▶ **PreToolUse / PostToolUse**

工具執行前後攔截、審查、改寫

- ▶ **HookMatcher 綁定**

事件 + 工具名(regex) → 回呼

- ▶ **回傳 dict 控制行為**

deny、注入脈絡、甚至中止任務

生命週期 Hooks

PreToolUse 攔下危險指令

main.py

```
async def block(input_data, tool_use_id, ctx):
    cmd = input_data.get("tool_input", {}).get("command", "")
    if "rm -rf" in cmd:
        return {"hookSpecificOutput": {
            "hookEventName": "PreToolUse",
            "permissionDecision": "deny",
            "permissionDecisionReason": "禁止 rm -rf"}}
    return {}

options = ClaudeAgentOptions(hooks={
    "PreToolUse": [HookMatcher(matcher="Bash", hooks=[block])])
```

外部 MCP 伺服器

社群寫好數百個 MCP 伺服器，掛上去就有。

- ▶ **三種傳輸**

stdio(本地子程序) / http / sse(遠端)

- ▶ **mcp_servers 設定**

用一個 dict 設定多個 server

- ▶ **同樣的命名規則**

mcp__server__tool，用全名放行

進程內與外部 MCP 可混用，一個 agent 同時掛兩種。

外部 MCP 伺服器

用 npx 掛上官方 filesystem MCP

```
main.py

options = ClaudeAgentOptions(
    mcp_servers={
        "fs": {
            "command": "npx",
            "args": ["-y",
                    "@modelcontextprotocol/server-filesystem", "."],
        },
    },
    allowed_tools=["mcp__fs__list_directory", "mcp__fs__read_file"],
)
```

PART 3 • 設定與狀態

設定與狀態

選模型、設人設、切換認證；讓對話跨輪次保存記憶。

08 系統提示・模型・認證

09 Session 持久化

系統提示 • 模型 • 認證

設定 agent 的人設、腦袋、身分三個維度。

- ▶ **system_prompt 三種寫法**

純字串 / preset / preset + append

- ▶ **model + fallback_model**

選模型；主模型過載時自動頂上

- ▶ **BYOK 靠環境變數**

Bedrock / Vertex / Azure，程式碼不用改

preset 會帶內建工具守則；純字串則完全自訂人設。

系統提示 • 模型 • 認證

preset + append 人設，並指定模型

```
main.py

options = ClaudeAgentOptions(
    system_prompt={
        "type": "preset",
        "preset": "claude_code",
        "append": "回答結尾附一個冷知識。",
    },
    model="claude-haiku-4-5",
    fallback_model="claude-sonnet-4-5",
)
```

Session 持久化

query() 預設無記憶；session 讓對話跨次呼叫延續。

- ▶ **抓 session_id**

從 SystemMessage(init).data 取得

- ▶ **resume 續談**

帶著完整上下文接回去

- ▶ **fork_session 分叉**

從某點長出平行支線，原 session 不變

Session 持久化

第一輪抓 id，之後 resume 接回去

main.py

```
async for message in query(prompt="記住數字 7", options=opts):
    if isinstance(message, SystemMessage) and message.subtype == "init":
        session_id = message.data["session_id"]

# 之後接回去，它還記得
await query(
    prompt="數字是多少？",
    options=ClaudeAgentOptions(resume=session_id),
)
```

PART 4 • 子代理

子代理

委派專職角色、把權限切乾淨、掛專屬工具箱、追蹤誰在做事。

10 自訂子代理

11 工具範圍

12 專屬 MCP

13 子代理追蹤

自訂子代理

把工作拆給專職角色：主 agent 像 PM，委派給子代理。

- ▶ **AgentDefinition 定義角色**

各有自己的人設、工具、模型

- ▶ **description 是觸發條件**

主 agent 靠它決定要不要委派

- ▶ **放行 "Agent" 工具**

委派透過內建 Agent 工具完成

AgentDefinition 是 camelCase；ClaudeAgentOptions 是 snake_case，別混。

自訂子代理

定義一個唯讀的 code-reviewer 子代理

```
main.py

options = ClaudeAgentOptions(
    agents={
        "code-reviewer": AgentDefinition(
            description="審查程式碼品質與安全性",
            prompt="你是資深 reviewer，找出 bug 與風險。",
            tools=["Read", "Grep", "Glob"],
            model="sonnet",
        ),
    },
    allowed_tools=["Read", "Grep", "Glob", "Agent"],
)
```

子代理工具範圍

最小權限：每個子代理只拿到剛好夠用的工具。

- ▶ **tools 白名單**

這個 agent 只能用這些

- ▶ **disallowedTools 黑名單**

明確禁止某些工具（camelCase！）

- ▶ **結構性限權**

被誘導也沒有工具可用

白名單框出大範圍，黑名單再戳掉個別危險項。

子代理工具範圍

編輯員能讀寫，但禁止跑 shell

main.py

```
"editor": AgentDefinition(  
    description="依指示修改檔案，但不執行任何指令。",  
    prompt="你負責編輯檔案。",  
    tools=["Read", "Write", "Edit"],  
    disallowedTools=["Bash"],    # camelCase !  
),
```

子代理專屬 MCP

讓每個子代理掛載自己專屬的工具箱。

- ▶ **全域先定義 server**

`options.mcp_servers = {...}`

- ▶ **mcpServers 引用**

指定這個 agent 掛哪幾個 (camelCase)

- ▶ **工具不互相污染**

各 agent 只看到該看的，選擇更準

子代理專屬 MCP

天氣助手只掛 weather server

main.py

```
options = ClaudeAgentOptions(  
    mcp_servers={"weather": weather_srv, "math": math_srv},  
    agents={  
        "weather-bot": AgentDefinition(  
            description="回答天氣問題。",  
            prompt="你只用 weather 工具查天氣。",  
            mcpServers=["weather"],  
            tools=["mcp__weather__get_weather"],  
        ),  
    },  
)
```

PARENT_TOOL_USE_ID

子代理追蹤

多代理的訊息混在一條流裡，怎麼分辨誰在說話？

- ▶ **parent_tool_use_id**
None = 主 agent；有值 = 某個子代理
- ▶ **SubagentStart / Stop**
hooks 標記子代理的開工與收工
- ▶ **還原活動軌跡**
兩者結合，做日誌或即時 UI

子代理追蹤

靠 parent_tool_use_id 分辨發話者

● ● ● main.py

```
def who(message):  
    pid = getattr(message, "parent_tool_use_id", None)  
    return "主 agent" if pid is None else f"子代理({pid[:8]})"  
  
options = ClaudeAgentOptions(hooks={  
    "SubagentStart": [HookMatcher(hooks=[on_start])],  
    "SubagentStop": [HookMatcher(hooks=[on_stop])],  
})
```

生產化

成本可觀測性、遙測、執行限制、安全沙箱、檔案系統記憶、進階持久化。

14 成本可觀測性

15 OTel 遙測

16 執行限制

17 安全沙箱

18 檔案系統記憶

19 進階持久化

成本與可觀測性

上生產線前，每次執行的成本、token、耗時都要能追。

- ▶ **ResultMessage 全算好了**

`total_cost_usd / usage / num_turns / duration_ms`

- ▶ **model_usage 揭露隱形成本**

子代理偷用貴模型也抓得出來

- ▶ **stderr callback**

攔截底層 CLI 的警告與除錯輸出

成本與可觀測性

任務收尾時印出一份執行報告

● ● ● main.py

```
elif isinstance(message, ResultMessage):  
    print(f"狀態 {message.subtype}")  
    print(f"花費 ${message.total_cost_usd:.6f}")  
    print(f"回合 {message.num_turns}")  
    print(f"耗時 {message.duration_ms} ms")  
    print(f"Token {message.usage}")
```

OpenTelemetry 遙測

要做組織級的長期監控，接 OpenTelemetry。

- ▶ **Claude Code 內建 OTel**

設好環境變數即可匯出

- ▶ **用 env 注入設定**

metrics 與 events 送到 OTLP 端點

- ▶ **接任何後端**

Langfuse / Grafana / Datadog / Honeycomb

想先看效果又不想架 collector？把 exporter 設成 console。

OpenTelemetry 遙測

用 env 打開內建遙測

main.py

```
options = ClaudeAgentOptions(env={
    "CLAUDE_CODE_ENABLE_TELEMETRY": "1",
    "OTEL_METRICS_EXPORTER": "otlp",
    "OTEL_LOGS_EXPORTER": "otlp",
    "OTEL_EXPORTER_OTLP_PROTOCOL": "grpc",
    "OTEL_EXPORTER_OTLP_ENDPOINT": "http://localhost:4317",
})
```

MAX_TURNS • MAX_BUDGET_USD

上下文與執行限制

別讓 agent 跑不停、燒錢、塞爆——三道閘門。

- ▶ **max_turns**

最多來回幾輪，防無限工具迴圈

- ▶ **max_budget_usd**

花費上限；超過時 subtype 變 error_max_budget_usd

- ▶ **betas 1M 上下文**

context-1m-2025-08-07

預算檢查在每次 API 呼叫後——是軟煞車，最終成本可能略超。

上下文與執行限制

三道閘門一起設，並偵測預算中止

main.py

```
options = ClaudeAgentOptions(  
    max_turns=2,  
    max_budget_usd=0.50,  
    betas=["context-1m-2025-08-07"],  
)  
  
if message.subtype == "error_max_budget_usd":  
    print("預算用完，提前停止")
```

安全沙箱

假設 prompt 可能被注入惡意指令，把 agent 關進沙箱。

- ▶ **cwd + add_dirs**

把家設在沙箱，白名單外的目錄碰不到

- ▶ **disallowed_tools**

整類高風險工具直接拿掉 (snake_case)

- ▶ **can_use_tool 把關路徑**

執行時用絕對路徑比對防穿越

縱深防禦：多道牆疊起來，任一道失守還有下一道。

安全沙箱

結構限權 + 執行時路徑把關

```
main.py

async def confine(tool, args, ctx):
    if tool in {"Write", "Edit"}:
        target = Path(args.get("file_path", "")).resolve()
        if not str(target).startswith(str(SANDBOX)):
            return PermissionResultDeny(message="超出沙箱")
    return PermissionResultAllow()

options = ClaudeAgentOptions(
    cwd=str(SANDBOX), disallowed_tools=["Bash"],
    permission_mode="default", can_use_tool=confine,
)
```

檔案系統設定與記憶

把專案知識寫進 CLAUDE.md，agent 每次啟動自動讀到。

- ▶ **setting_sources** 控制載入
user / project / local
- ▶ **CLAUDE.md = 靜態記憶**
手寫、版本控管的長期知識
- ▶ **也載入 commands / agents / skills**
整個 .claude/ 目錄

不設 setting_sources 會載入全部來源；要乾淨環境就明確指定或用 []。

檔案系統設定與記憶

載入專案層級設定與記憶

main.py

```
options = ClaudeAgentOptions(  
    cwd=str(HERE),  
    setting_sources=["project"],    # 載入 .claude/ + CLAUDE.md  
)  
  
if isinstance(message, SystemMessage) and message.subtype == "init":  
    print(message.data.get("slash_commands"))  
    print(message.data.get("agents"))
```

進階持久化與壓縮

長時間運作的 agent：壓縮時保住記憶、把 session 存到任何後端。

- ▶ **PreCompact hook**

壓縮歷史前，注入「務必保留 X」

- ▶ **session_store 換後端**

InMemory / Redis / Postgres / S3

- ▶ **把整個系列串起來**

hooks + session + 限制 + 記憶

進階持久化與壓縮

壓縮前保護關鍵記憶 + 自訂後端

main.py

```
async def protect(input_data, tool_use_id, ctx):
    return {"hookSpecificOutput": {
        "hookEventName": "PreCompact",
        "additionalContext": "壓縮時務必保留：需求、決定、待辦。"}}

options = ClaudeAgentOptions(
    session_store=InMemorySessionStore(),
    hooks={"PreCompact": [HookMatcher(hooks=[protect])]},
)
```

RECAP

你學會了什麼

▶ ① 基礎

query() 訊息流、訊息型別、即時串流

▶ ② 工具與權限

can_use_tool、@tool 自訂工具、Hooks、外部 MCP

▶ ③ 設定與狀態

system_prompt / model / BYOK、Session 續談與分叉

▶ ④ 子代理

AgentDefinition 委派、工具範圍、專屬 MCP、追蹤

▶ ⑤ 生產化

成本、OTel 遙測、執行限制、安全沙箱、記憶、壓縮

THANK YOU

開始打造你的 Agent

19 課的 main.py 都能直接跑、改、組合。挑一個主題深入，或把它做成影片。

- 官方文件 code.claude.com/docs/zh-TW/agent-sdk/overview
- Python API code.claude.com/docs/zh-TW/agent-sdk/python
- 官方範例 github.com/anthropics/claude-agent-sdk-python

